

Experiment-05

Aim:

- A. A website maintains records of users who have registered on the platform and the status of confirmation requests sent to them. A confirmation request can either be successfully completed or may expire without completion.
If a user has never received any confirmation request, their confirmation rate should be considered **0**. Display the user ID along with the confirmation rate rounded to **2 decimal places**. The result can be returned in any order.
- B. A ride-booking company maintains information about trip requests and the users involved in those trips. Some users may be restricted from using the platform, while others are active participants. Each trip can either be completed successfully or cancelled by one of the participants.
For each day between **2013-10-01** and **2013-10-03**, determine the proportion of cancelled trips among all valid trips. A trip should be considered valid only when both the customer and the driver associated with that trip are active users. The final result should display the date and the calculated cancellation rate rounded to two decimal places.

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Theory:

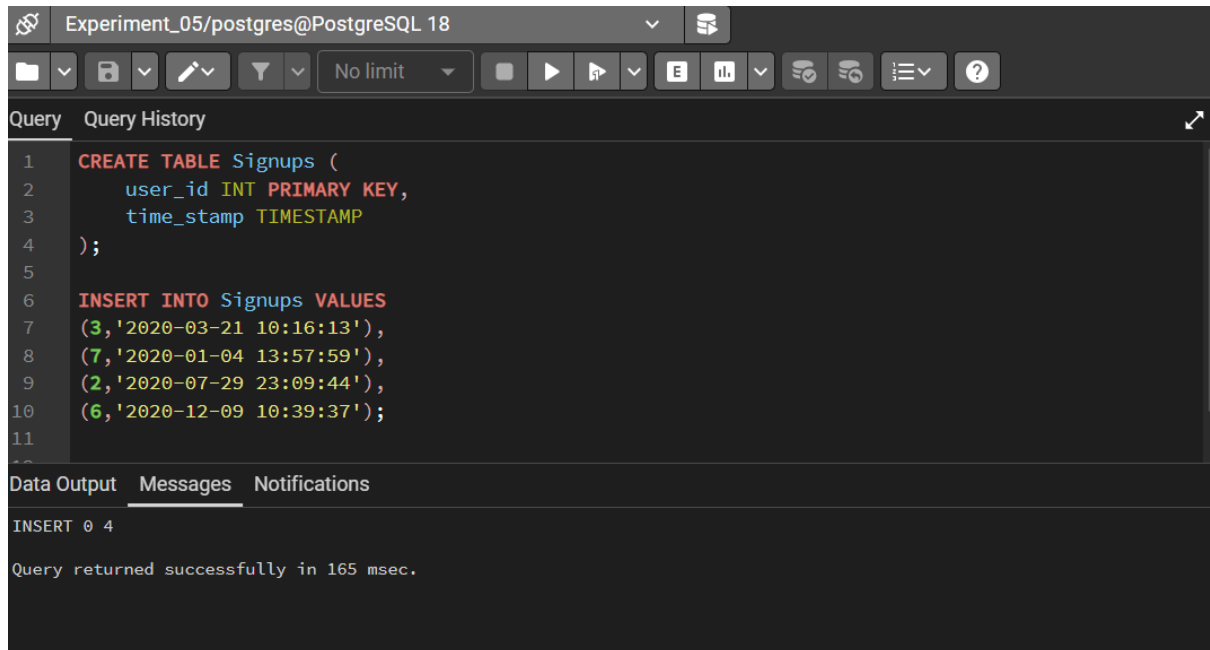
The CASE expression is a conditional expression used to return different values depending on one or more conditions. It is commonly used in:

- SELECT statements
- UPDATE statements
- ORDER BY
- WHERE clauses

It is similar to an if-else-if ladder or a switch-case statement in programming languages.

Code and Output:

A.

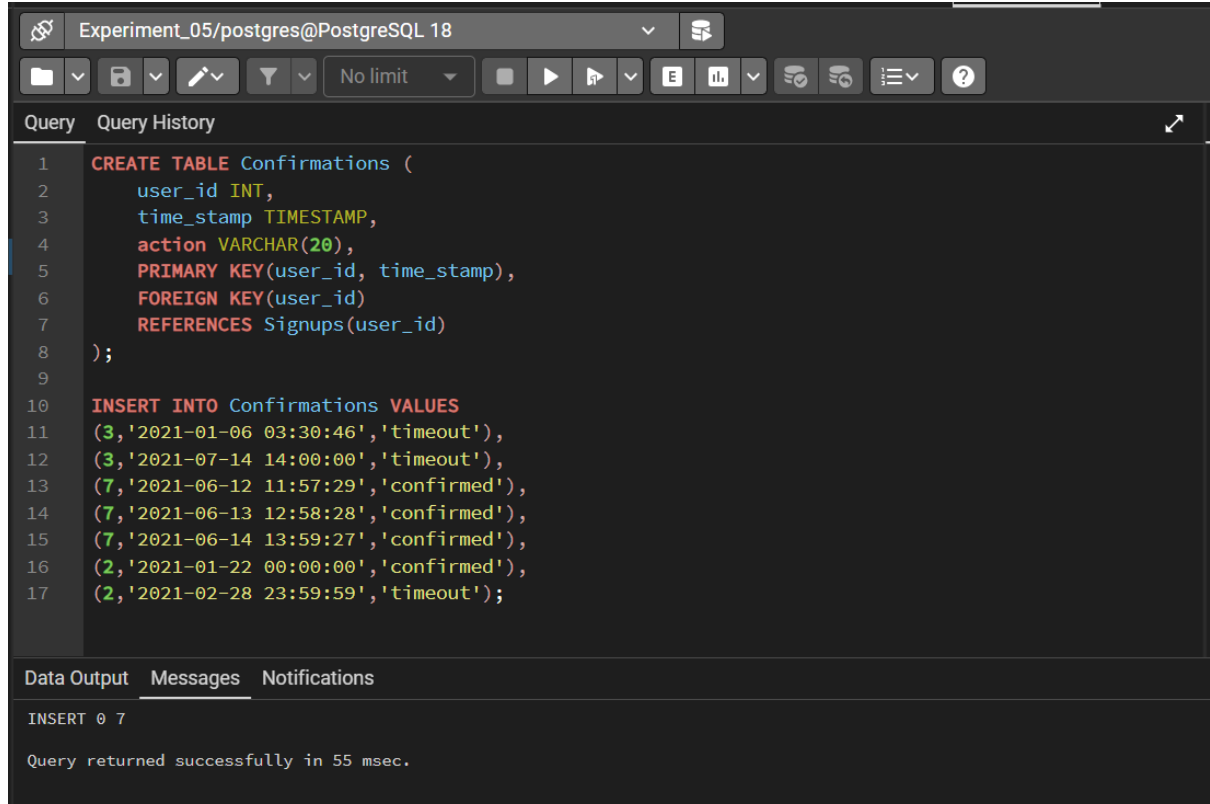


The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'Experiment_05/postgres@PostgreSQL 18'. The 'Query' tab is active, displaying the following SQL code:

```
1 CREATE TABLE Signups (  
2     user_id INT PRIMARY KEY,  
3     time_stamp TIMESTAMP  
4 );  
5  
6 INSERT INTO Signups VALUES  
7 (3,'2020-03-21 10:16:13'),  
8 (7,'2020-01-04 13:57:59'),  
9 (2,'2020-07-29 23:09:44'),  
10 (6,'2020-12-09 10:39:37');  
11
```

Below the query editor, the 'Messages' tab is active, showing the execution results:

```
INSERT 0 4  
  
Query returned successfully in 165 msec.
```

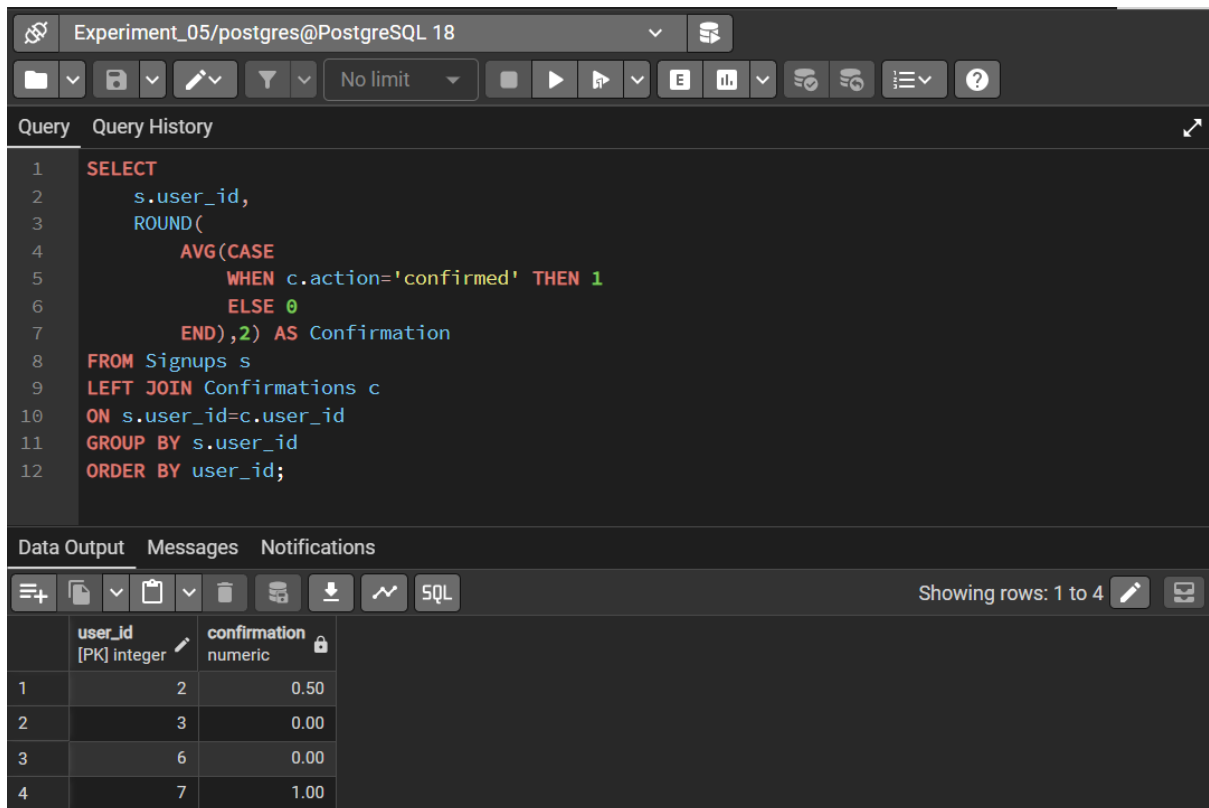


The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'Experiment_05/postgres@PostgreSQL 18'. The 'Query' tab is active, displaying the following SQL code:

```
1 CREATE TABLE Confirmations (  
2     user_id INT,  
3     time_stamp TIMESTAMP,  
4     action VARCHAR(20),  
5     PRIMARY KEY(user_id, time_stamp),  
6     FOREIGN KEY(user_id)  
7     REFERENCES Signups(user_id)  
8 );  
9  
10 INSERT INTO Confirmations VALUES  
11 (3,'2021-01-06 03:30:46','timeout'),  
12 (3,'2021-07-14 14:00:00','timeout'),  
13 (7,'2021-06-12 11:57:29','confirmed'),  
14 (7,'2021-06-13 12:58:28','confirmed'),  
15 (7,'2021-06-14 13:59:27','confirmed'),  
16 (2,'2021-01-22 00:00:00','confirmed'),  
17 (2,'2021-02-28 23:59:59','timeout');
```

Below the query editor, the 'Messages' tab is active, showing the execution results:

```
INSERT 0 7  
  
Query returned successfully in 55 msec.
```



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'Experiment_05/postgres@PostgreSQL 18'. The 'Query' tab is active, displaying a SQL query that calculates the average confirmation rate for each user. The query is as follows:

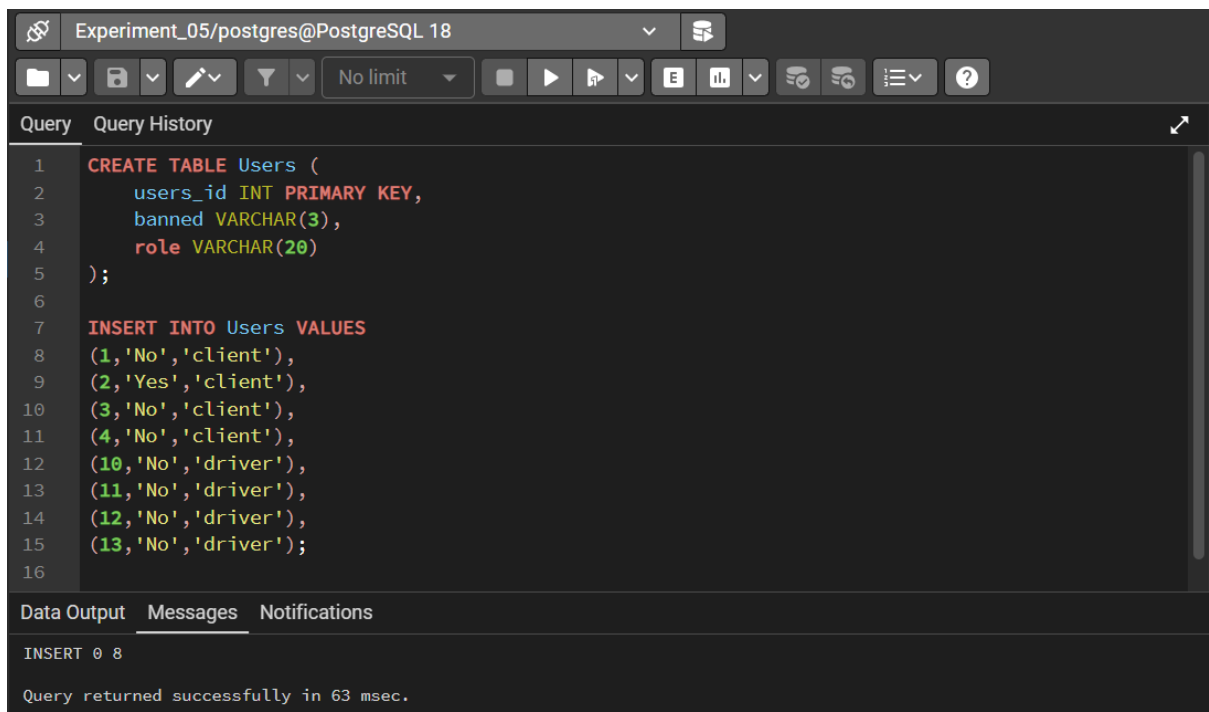
```
1 SELECT
2     s.user_id,
3     ROUND(
4         AVG(CASE
5             WHEN c.action='confirmed' THEN 1
6             ELSE 0
7         END),2) AS Confirmation
8 FROM Signups s
9 LEFT JOIN Confirmations c
10 ON s.user_id=c.user_id
11 GROUP BY s.user_id
12 ORDER BY user_id;
```

Below the query, the 'Data Output' tab shows the results of the query. The results are displayed in a table with two columns: 'user_id' (integer, primary key) and 'confirmation' (numeric). The table contains four rows of data:

	user_id [PK] integer	confirmation numeric
1	2	0.50
2	3	0.00
3	6	0.00
4	7	1.00

The interface also includes a 'Query History' tab and a 'Messages' tab. The 'Data Output' tab shows 'Showing rows: 1 to 4'.

B.



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'Experiment_05/postgres@PostgreSQL 18'. The 'Query' tab is active, displaying a SQL query that creates a table named 'Users' and inserts data into it. The query is as follows:

```
1 CREATE TABLE Users (
2     users_id INT PRIMARY KEY,
3     banned VARCHAR(3),
4     role VARCHAR(20)
5 );
6
7 INSERT INTO Users VALUES
8 (1,'No','client'),
9 (2,'Yes','client'),
10 (3,'No','client'),
11 (4,'No','client'),
12 (10,'No','driver'),
13 (11,'No','driver'),
14 (12,'No','driver'),
15 (13,'No','driver');
```

Below the query, the 'Data Output' tab shows the results of the query. The results are displayed in a table with two columns: 'user_id' (integer, primary key) and 'confirmation' (numeric). The table contains four rows of data:

	user_id [PK] integer	confirmation numeric
1	2	0.50
2	3	0.00
3	6	0.00
4	7	1.00

The interface also includes a 'Query History' tab and a 'Messages' tab. The 'Data Output' tab shows 'Showing rows: 1 to 4'.

```
Experiment_05/postgres@PostgreSQL 18
Query Query History
1 CREATE TABLE Trips (
2     id INT PRIMARY KEY,
3     client_id INT,
4     driver_id INT,
5     city_id INT,
6     status VARCHAR(30),
7     request_at DATE
8 );
9
10 INSERT INTO Trips VALUES
11 (1,1,10,1,'completed','2013-10-01'),
12 (2,2,11,1,'cancelled_by_driver','2013-10-01'),
13 (3,3,12,6,'completed','2013-10-01'),
14 (4,4,13,6,'cancelled_by_client','2013-10-01'),
15 (5,1,10,1,'completed','2013-10-02'),
16 (6,2,11,6,'completed','2013-10-02'),
17 (7,3,12,6,'completed','2013-10-02'),
18 (8,2,12,12,'completed','2013-10-03'),
19 (9,3,10,12,'completed','2013-10-03'),
20 (10,4,13,12,'cancelled_by_driver','2013-10-03');
```

```
Experiment_05/postgres@PostgreSQL 18
Query Query History
1 SELECT
2     t.request_at AS Day,
3     ROUND(AVG(CASE
4         WHEN t.status IN
5             ('cancelled_by_driver', 'cancelled_by_client') THEN 1
6         ELSE 0
7     END), 2) AS Cancellation_rate
8 FROM Trips t
9 JOIN Users c ON t.client_id = c.users_id
10 JOIN Users d ON t.driver_id = d.users_id
11
12 WHERE c.banned = 'No'
13     AND d.banned = 'No'
14     AND t.request_at BETWEEN '2013-10-01' AND '2013-10-03'
15
16 GROUP BY t.request_at;
```

Data Output Messages Notifications

Showing rows: 1 to 3

	day date	cancellation_rate numeric
1	2013-10-01	0.33
2	2013-10-02	0.00
3	2013-10-03	0.50

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with Switch-Case Statements in PostgreSQL.
- Got hands on experience with integrating subqueries and case statement in PostgreSQL
- Practiced commands of Switch-Case in SQL.